

BeliefLens: Belief Dynamics in LLM Forecasting Harnesses

Recovering the update rule, and what changes when the harness is real

BeliefLens Project

Deep Digital Intelligence · <https://github.com/DDI-droid/beliefLens>

September 11, 2026 · v2.0

Abstract

LLM forecasters are *judgmental* forecasters: behind every stated probability is something that behaves like a belief, and behind every revision an update rule. BeliefLens measures both. We build a date-gated replay sandbox over 339,801 news articles and run forecasting harnesses under two protocols—**sequential** (five dates in order, own history carried, $k=5$ rollouts) and **parallel** (fresh one-shot per date, no history, $k_2=5$ rollouts)—so that every belief is a distribution and the effect of carrying history is isolated by construction. From each trajectory we *recover the reasoning as a literal-free program*: named variables extracted from the harness’s own reasoning are the only values a synthesized program may use, and blind sample and temporal holdouts decide whether the recovered method is real. The recovered program then serves as an executable referee for counterfactual probes. We run this twice. First on four *prompt-defined reasoning shapes* on one shared tool loop, where we find a stability–plasticity inversion, a genuinely fixed log-odds update formula for the Bayesian shape (its early-days program predicts held-out late days at MAE ≈ 0.003), a three-way decomposition of the FutureSim shape’s rigidity into acquisition, procedure and history—but *not* rule violation—and the incompressibility of free-form ReAct reasoning. Then we run it again with the **real orchestration code** of two source systems: Analytica’s analyzer/grounder/synthesizer pipeline with its linear composition computed in code, and the BLF system’s Algorithm 1 with K -trial shrunken logit-mean aggregation. The real systems are markedly more rigid than their prompt shapes—Analytica’s update gain falls from 0.22 to 0.05 and its total belief movement by 3.5 $\times$; BLF’s falls from 0.47 to -0.17 —and in both cases the rigidity is traceable to specific architecture, not to an instruction a model may decline. All 1,432 logged searches replay with zero articles past the date gate, and an independent adversarial review is incorporated with every correction documented.

1 Introduction

Benchmarks such as FutureSim [2] score LLM forecasting agents on outcomes, and observe behaviours—anchoring to initial forecasts, “self-conditioning” on past rationales—that are claims about *belief dynamics*. But a score cannot say where a pathology lives: in what the agent *believes*, in how it *updates*, or in how it *gathers evidence*. Those are three different failures with three different fixes, and one number per condition cannot separate them. This report develops instrumentation that does.

The central device is **program recovery** (§9). If a harness’s day-to-day reasoning is a stable method applied to changing inputs, then a single program—with *no numeric literals anywhere*, only named quantities extracted from the reasoning itself—should reproduce every forecast the harness made, with only the variable bindings changing between days. The program is then the *reasoning structure*; the bindings are the *belief state*; and the split is forced by construction, because the literal ban leaves a number nowhere to hide. It must arrive as a semantically named input or be computed.

Once such a program exists, three things become measurable that were not. Belief **distance** becomes a distance in the program’s own coordinates rather than between two output probabilities. **Theory change** becomes visible as the program failing to fit later days. And the program becomes an **executable referee**:

inject one piece of news, ask what the agent’s own rule implies, and compare that against what the agent actually did (§10). An agent can be caught violating itself.

Contributions.

- **Infrastructure.** A replayable forecasting environment on the FutureSim news corpus (339,801 articles, 2025-12-01–2026-03-31) with an SQL-enforced date gate verified by *replaying all 1,432 logged searches* (zero breaches); dual elicitation protocols (sequential $k=5$, parallel $k_2=5$) yielding belief *distributions*, not points; and a hard per-run call budget enforced in the model wrapper.
- **Method.** A literal-free program space with a mechanical anti-gaming validator (no literals, no boolean constants, no constant-only arithmetic, no value-spelling names such as `two=2`, no dead code, no x/x), an answer-copy filter over raw/odds/percent/complement transforms, and blind sample and temporal holdouts as the deciding gate; plus a standing audit that re-verifies all of it against the collected data on demand.
- **Findings, prompt tier.** A stability–plasticity inversion across harnesses; recovery of a true fixed update formula for the Bayesian shape; a three-way decomposition of the FutureSim shape’s rigidity; quantified conservatism (update gains 0.22–0.47, all < 1); and the incompressibility of free-form ReAct reasoning.
- **Findings, real tier.** The same replay experiment run with faithful re-implementations of two source systems’ *orchestration code* (§8). The prompt-tier results *understate* rigidity, because the architectures enforce in code what a prompt can only request. The shape-versus-real difference is itself a measurement: how much of a system’s belief dynamics lives in its wiring rather than in its instructions.

2 Background: the source systems in their own terms

Because this report both imitates these systems (§6) and runs two of them (§8), we state their base forms precisely enough that the deltas are checkable.

Analytica [1] — the architecture. Three agent roles, two algorithms. ANALYZE (their Alg. 1) starts from a root proposition ρ_0 and repeatedly calls an *Analyzer* A_A to add child nodes until a leaf budget L_{max} or a completion signal is reached. SYNTHESIZE (their Alg. 2) walks the tree bottom-up: *Grounder* agents A_G evaluate all leaves *in parallel*, each returning a soft truth value and a written report; a *Synthesizer* A_S then scores each non-leaf from its children under a **linear** rule,

$$\rho_i \cdot p_{true} = \beta_0 + \sum_j \beta_j \bar{\rho}_{ij} \cdot p_{true},$$

where the LLM emits the β_j and β_0 as JSON and the arithmetic is not the model’s job. Their §4.2 justifies this with a bias/variance argument: verify simple leaves, and let an “effectively free mathematical wrapper” aggregate them, because a linear rule dampens leaf noise where a fuzzy-logic rule amplifies it. Crucially for us, the paper already defines **resynthesis** (their §4.1): after a tree is grounded, a user may “edit the truth value, statements, or reports of any node, or add/remove nodes” and the system “update[s] only the affected branches up to the root.” Continual forecasting (§8) is exactly that mechanism with the counterfactual replaced by a calendar.

FutureSim [2] — the environment. A chronological, replayable simulation on a daily cadence. Task state is a CSV, one row per question (background, resolution criteria, resolution date, latest forecast); ground truth is appended once the simulated date passes resolution. The context is the set of CCNews documents available by that date, chosen because its snapshots are reliably dated and not retroactively edited. The environment enforces exactly two actions, `submit_forecast(question_id, outcomes)` and `next_day()`; tools, prompts and orchestration are the agent’s business. Their benchmark is 330 questions resolving 2026-01-01–03-28 over 7.36M articles with a hybrid semantic+keyword search tool returning five 512-token chunks. Two of their findings are this report’s direct parents:

- **Anchoring.** Started from a deliberately poor initial forecast set, every frontier agent failed to adapt away from it—not one reached even the Brier skill score of making no prediction at all, *despite being told* its predictions were scoring negatively.
- **Sequential < direct.** The same agent, asked each question directly one day before resolution with full information and no prior prediction to anchor to, scored 31.2% accuracy; walking the same information forward day by day, it scored 24.8%.

That second result is a single aggregate accuracy gap. It establishes the phenomenon but cannot localise it. Our SEQ/PAR design (§6) is that experiment refactored into a per-date, per-rollout *distribution*, so the gap decomposes into anchoring, conservatism, lag and placement components that are separately measurable.

Agent-BRACE [3] and BLF [5] — the representation. Agent-BRACE formalises the agent as a POMDP and decouples it into a *belief state model* and a *policy model*, trained jointly with reinforcement learning; the belief is a set of atomic natural-language claims, each tagged on an ordinal Words-of-Estimative-Probability scale (*confirmed* > *almost certain* > *probable* > *possible* > *unlikely* > *doubtful* > *unknown*). It is therefore a *training method*, not an installable harness—without the RL stage there is nothing to run—so it contributes the framing question (is the externalised structure the working belief?) while the harness we implement comes from BLF, which instantiates the same idea for forecasting. BLF’s Algorithm 1:

```
Require: question q, cutoff date d, max steps T
b0 <- initial belief (p = 0.5); m0 <- (q, b0)
for t = 1 .. T:
  (a_t, b_t) <- LLM(m_{t-1})          # update belief AND choose action, one generation
  if a_t = submit(p_hat): return p_hat
  o_t <- Env(a_t; q, d)
  if a_t is web search: o_t <- LeakFilter(o_t, d)    # 2nd-pass LLM filter
  m_t <- m_{t-1} (+) (a_t, o_t, b_t)
Force submit; return b_T.p
```

The belief b_t is a semi-structured JSON object holding a probability, a confidence level, key evidence for and against, and open questions. Around the loop sit $K=5$ independent trials aggregated by a shrunk logit-space mean $\hat{p} = \sigma(\alpha \cdot \frac{1}{K} \sum_k \text{logit}(p_k))$ with $\alpha \in [0, 1]$ falling as between-trial variance rises, and hierarchical Platt calibration with per-source intercept offsets. $T_{max}=10$; submitted probabilities are clamped to $[0.05, 0.95]$ to bound worst-case Brier loss.

ReAct [4] supplies the floor: an interleaved think/act loop with no imposed structure. Unlike the other three it *is* a prompt pattern over a tool loop, so our **react** coincides with its source and needs no “full” variant.

3 Experimental Infrastructure

3.1 Corpus and date gate

Articles are the FutureSim corpus (HuggingFace **shash42/forecast-news**; CCNews-derived, deduplicated, dated), sliced to the simulation window: 120 day-partitions, 2025-12-01–2026-03-31, indexed to SQLite FTS5 (BM25), 339,801 rows. At simulation date D the one search tool returns only articles with date $\leq D$; **the constraint is in the SQL, not in the prompt**. *Verification*: the audit re-executes every search any harness ever logged (483 in the main runs, 949 in the scale-up) with its own turn’s date; zero returned articles postdate the gate.

We do not use FutureSim’s 48 GB prebuilt hybrid index. Retrieval here is BM25 keyword search, weaker than the paper’s, but byte-identical across every harness—so it is a constant of the experiment rather than a confound in it.

3.2 Model choice and contamination

Harnesses run on `gpt-5-mini` (released 2025-08-07). This is a validity decision, not a cost decision: the questions resolve January–March 2026, so any later model would *recall* rather than forecast, and a model that already knows the outcome does not move its belief—destroying the very thing being measured. The extractor (`gpt-5.2`) post-dates the events; it writes variables and programs but makes no forecasts, and its one leakage channel (tilting extracted values toward the known outcome) is audited via the preserved pre-reconciliation variable tables: across 320 tables, exactly one systematically answer-tracking variable was found (`posterior_odds`, odds-form, 10/10 instances), and no recovered program references it.

3.3 Questions

The paired control: *Will the USA / will Canada win the Milano-Cortina men’s hockey gold?* One evidence stream, both resolving YES/NO. Both teams won their semifinals on 02-20—the last simulated date—and the final resolved 02-22, *outside* the window.

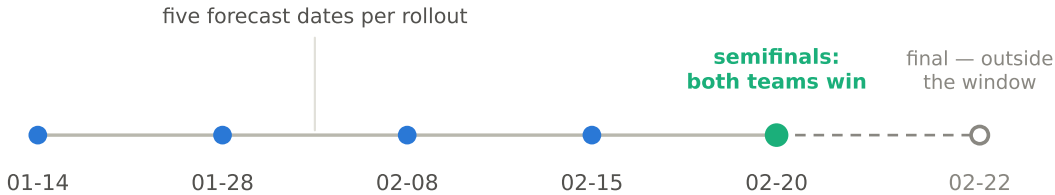


Figure 1: The simulated calendar. Five forecast dates per rollout; the gold medal is decided after the last date any harness ever sees.

That event path determines which metrics are legitimate. A correct forecaster on 02-20 raises *both* beliefs toward ~ 0.5 , so the proper measures are coherence measures—both beliefs rising across the semifinal (*semi-rise*) and $P_{\text{USA}} + P_{\text{CAN}} \rightarrow 1$ (*sum@last*)—and *not* directional separation. Our original design scored directional separation; the adversarial review (§11) showed it would have punished correct forecasting, and it was replaced before any conclusion was drawn from it. A scale-up set of four further questions from the OpenForesight dataset (FutureSim’s question source) supports the recovery statistics of §9; all four resolve YES, a bias we disclose and therefore exclude from every directional claim.

4 Harnesses: two tiers

4.1 Tier A — four prompt-defined reasoning shapes

All four Tier-A harnesses run on *one* shared code path: a single-agent tool loop (at most five `search_news` rounds, then a forced answer, output contract `FORECAST: p`). What differs per harness is (i) the system prompt imposing the reasoning structure and (ii) a small amount of shape-specific code—for `futuresim` and `bayesian`, programmatic extraction of the memory or belief block and its re-injection the next day. The prompts are reproduced verbatim in Appendix B.

What Tier A is, and is not. These are *prompt-level re-implementations of each source system’s reasoning shape*, not the source systems. Real Analytica builds its tree and computes its linear composition *in orchestration code*, with parallel per-leaf grounder agents. The real FutureSim baseline agent (`extttbasicsAgent`) queries the question DataFrame by writing Python in a sandbox (`extttquery_df`), manages a structured memory store through four CRUD tools, submits a *distribution* over outcomes it generates itself, and decides for itself when to advance the clock with `extttnext_day`. The real BLF system adds multi-trial aggregation and calibration. Holding the substrate fixed is a deliberate control—installing three repositories would compare repositories, whereas fixing the substrate makes reasoning shape the only manipulated

harness	structure imposed	carries across days	lineage
react	free think/search loop	forecast history*	ReAct [4]
futuresim	6 ordered guidelines; base rate first; forced ≤ 200 -word memory update	history + memory block	FutureSim baseline harness [2]
analytica	decompose 3–5 sub-propositions $\rightarrow$ ground each by search $\rightarrow$ synthesize with weights	forecast history*	Analytica [1]
bayesian	prior $\rightarrow$ evidence $\rightarrow$ per-item likelihood ratio $\rightarrow$ posterior; forced belief block	history + belief block	BLF [5] / Agent-BRACE [3]

Table 1: The four reasoning shapes. *All four see their own prior forecasts in the sequential protocol (the anchoring pressure, applied uniformly); **react** is therefore a *history-anchored* floor, not a memoryless one.

variable—but it bounds every Tier-A claim to being about *shapes at fixed capacity*. §8 removes that bound for two of the three systems and treats the difference as data.

4.2 Tier B — real orchestration code

Two source systems were re-implemented as what they actually are: multi-agent control flow, structured state and composition arithmetic executed in Python, with the LLM confined to the roles the papers give it. They run under the identical protocols of §6—same model, corpus, date gate, questions, $k=k_2=5$.

analytica-full. An *analyzer* agent decomposes the question into 3–5 soft sub-propositions (leaf budget 8, up to 3 expansion rounds on day 1); parallel per-leaf *grounder* agents each generate search queries, read gated articles, and conclude a leaf probability with a written report in a separate second call; a *synthesizer* agent assigns weights; and the linear composition $p = \text{clamp}(\beta_0 + \sum_i \beta_i p_i)$ is computed recursively *in code*—the model never states the final probability.

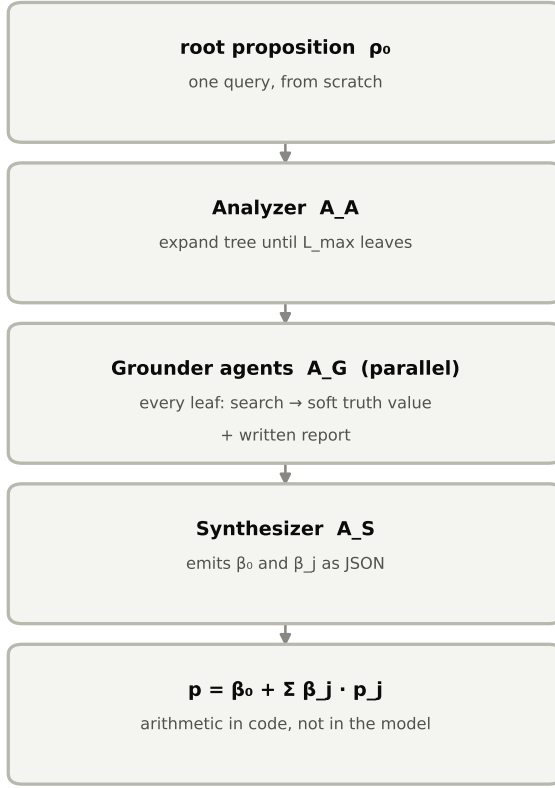
The paper is single-shot, so continual forecasting requires an extension, and the extension is precisely the paper’s own resynthesis mechanism (§2) driven by a calendar:

1. The proposition tree is carried as state rather than rebuilt.
2. Each new date runs an analyzer **edit pass**—add or remove leaves against the new window. Its prompt states explicitly that most days need no change.
3. Every leaf is re-grounded in parallel, but **delta-grounded**: each grounder’s search window is restricted to the period since *that leaf* was last grounded, and the grounder is shown its own previous probability and report, with the instruction to keep its value close if nothing material changed.
4. Synthesizer weights are re-elicited **only** where a node’s child set changed; otherwise the stored β are reused.
5. The composition is recomputed in code.

Three lines of architecture therefore produce rigidity by construction, and each is nameable: a leaf that sees only news *since it last looked* cannot react to standing evidence it has already discounted; a leaf handed its own prior value is anchored by design; and a parent whose children did not change reuses yesterday’s weights outright. None of this is an instruction the model may decline.

blf-full. The BLF system’s Algorithm 1 verbatim in structure: the belief is an explicit JSON state (probability, confidence, evidence for/against, open questions); each step one generation returns an action (search or submit) *and* the updated belief; at most $T_{\max}=10$ steps; the submitted probability is clamped to $[0.05, 0.95]$. $K=3$ trials run per day (the paper uses $K=5$; reduced for budget) and are aggregated by the paper’s shrunken logit-space mean with $\alpha = 1/(1 + \text{Var}(\text{logit}(p_k)))$ —our instantiation of their variance-dependent α , whose functional form they do not pin down. The belief state of the median-probability trial is carried across days in SEQ and rebuilt from scratch in PAR. The paper’s **LeakFilter** (a second-pass LLM filter on search results) is replaced by our SQL date gate, which is a strictly stronger guarantee. Platt calibration is omitted: it requires resolved training questions, which the sandbox window does not contain.

Analytica as published — one pass



As run here — one pass per simulated date

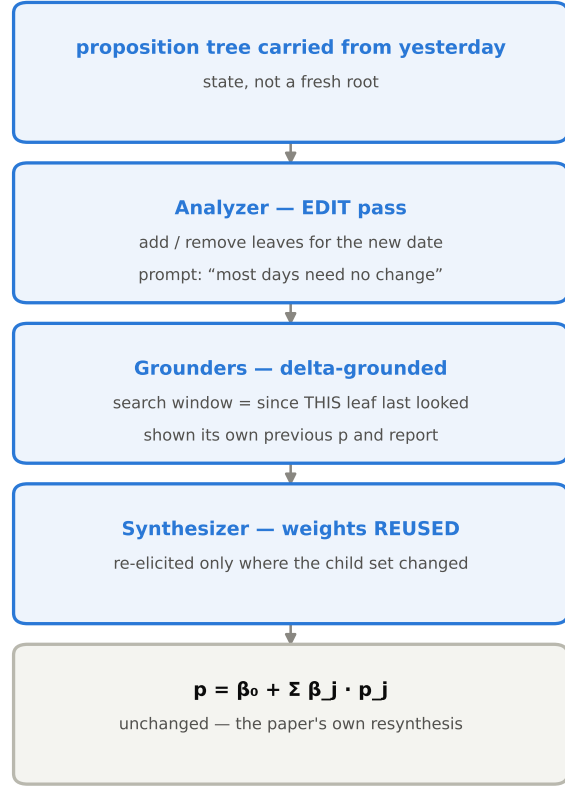


Figure 2: Analytica: as published, and as run here. Highlighted boxes are the whole continual-forecasting delta; everything else, including the composition arithmetic, is the paper’s machinery unchanged.

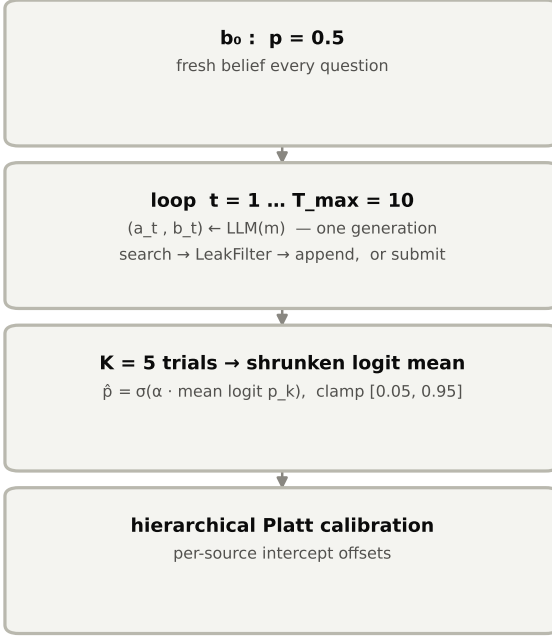
Cost discipline. Every model call passes through a wrapper that logs tokens and raises `BudgetExceeded` at a hard cap, so no run can overshoot silently. The Tier-B run cost 3,229 calls (6.39M input, 1.46M output tokens, 41 minutes at 4 workers) against a 4,000 cap, with zero turn errors in 200 forecasts. The full ledger is Appendix D.

FutureSim’s simulator. The third source system is not re-implemented but *integrated*: their actual code-base at a pinned commit, plus two adapter files (a direct OpenAI inference provider, and an implementation of their `BaseSearchTool` over our gated index) and a 71-line patch to their runner. A zero-cost structural dry run (`--no_inference`) confirms that the custom dataset loads, both questions activate, the BM25 backend serves all 339,801 articles, exact answer matching is selected—so no matcher model is needed—and the day loop enters. Leaving `articles_base` empty is deliberate: corpus staging becomes a no-op, so the agent’s *only* information channel is the search tool. Their `--start_date/--end_date` define the *resolution* window, not the simulated days. The live run is reported as pending in §12.

5 The Literal-Free Program Space

A recovered program is one Python function `forecast()` over an injected namespace, with: **(R1)** no numeric, string, or *boolean* literals anywhere; **(R2)** no user-defined functions, imports, attributes, subscripts, comprehensions, or containers (recursion and `while` permitted; execution carries a 200k-step budget so non-termination is a failed fit); **(R3)** calls only to eleven whitelisted operations (`clamp`, `logistic`, `noisy_or`,

BLF as published — one forecast date



As run here — belief carried across dates

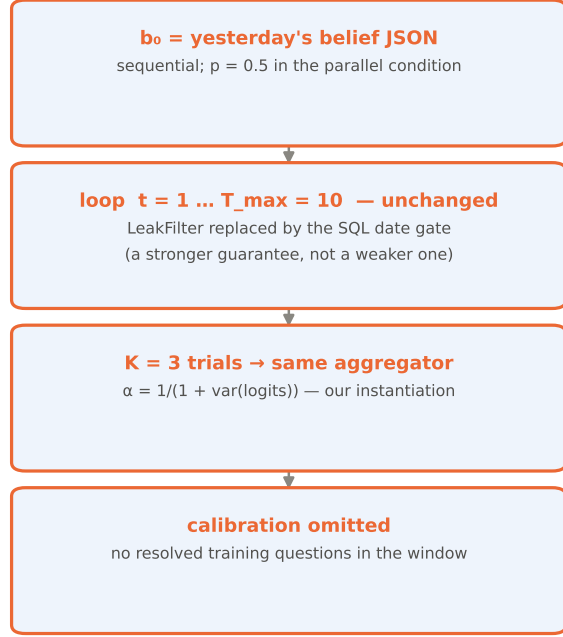


Figure 3: BLF: as published, and as run here. Three deltas—belief carried across dates, $K=3$ rather than 5, and no calibration stage—each disclosed in §12.

odds, prob, min/max/abs/round/ exp/log/sqrt) and nine named constants; **(R4)** *no value may hide in a name*: two=2, half=0.5, const_0_62 are rejected in variable tables, schemas, and program locals alike; **(R5)** no constant-only arithmetic (TENTH*(TWO+TWO+TWO)=a smuggled 0.6), no self-ops ($x-x$, x/x), no dead code (ZERO*(...), identity ops, unused locals).

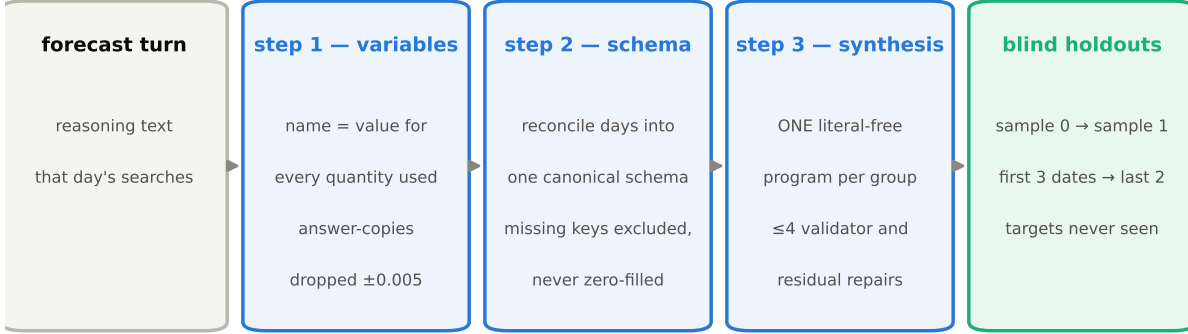
Every one of R4 and R5 exists because the first version of the rule was broken by the adversarial reviewer (§11). An eleven-exploit battery must reject under the validator and does; a legitimate program passes. Because in-sample fit with visible targets cannot by itself distinguish structure from memorisation, **the deciding gate is out-of-sample**: programs are re-synthesized on a subset whose complement’s targets the synthesizer never sees.

6 Experiment 1: Replay

6.1 Protocols

Sequential (seq): five dates in order (01-14, 01-28, 02-08, 02-15, 02-20); each day the harness sees its own previous forecasts (and its carry block) and may search the gated corpus; $k=5$ independent rollouts of the whole trajectory. **Parallel (par)**: for each date, a fresh single-shot run given all information up to that date and *no* history; $k_2=5$ rollouts per date. Day 1 of a sequential rollout is, by construction, a parallel draw—a built-in sanity check (Tier A: 12/16 day-1 sequential values inside the day-1 parallel spread at ± 0.05 ; Tier B: 4/4).

All 760 collected forecasts parse under the strict parser (200 sequential + 200 parallel on the pair, 160 scale-up, 200 real-tier; no fallback; percent and comma handled; out-of-range triggers a retry), and every stored forecast re-parses identically from its own turn text.



The literal ban forces the split: a number is either a named input or a computation. The blind holdouts, not the fit, are the gate.

Figure 4: The recovery pipeline. One program per harness–question, then two blind re-syntheses whose held-out targets the synthesizer never sees.

6.2 Metrics

For dates $t \geq 2$: **anchor gap** = $\text{SEQ}_t - \overline{\text{PAR}}_t$ (signed; what carrying history does relative to evidence alone); **lag advantage** = $|\text{SEQ}_t - \overline{\text{PAR}}_t| - |\text{SEQ}_t - \overline{\text{PAR}}_{t-1}|$ (positive $\Rightarrow$ the sequence tracks *yesterday's* evidence-conditioned belief better than today's); **update gain** = OLS slope of ΔSEQ_t on $\Delta\overline{\text{PAR}}_t$ (1 = moves with the evidence; <1 conservatism; <0 moves against it); **placement** = percentile of each SEQ forecast within that date's PAR draws (50 = unbiased); **stabilisation** = $\text{sd}(\text{SEQ})/\text{sd}(\text{PAR})$ per date; **coherence** = $P_{\text{USA}} + P_{\text{CAN}}$ (**sum@last** is its value on 02-20, **semi-rise** the fraction of rollouts in which both paired beliefs rose across the semifinal); **total movement** $|\Delta|_{\text{tot}}$, the summed absolute day-to-day change of a rollout; and **Brier** [6]. Placement is a probability-integral-transform (PIT) statistic, and PIT_{med} its median.

Throughout this report, Brier is the plain quadratic score $\frac{1}{n} \sum_i (p_i - y_i)^2$ with $y_i \in \{0, 1\}$ the resolved outcome, pooled over the pair—not a skill score (no baseline subtraction, no 1–MSE orientation, no difficulty adjustment); 0 is perfect, and **0.25** is the constant-0.5 forecaster.

7 E1 Results: the prompt tier

harness	replay (SEQ, $k=5$)			SEQ vs PAR				
	$ \Delta _{\text{tot}}$	semi-rise	sum@last	gap	gap	gain	lag adv.	PIT_{med}
analytica	0.331	0.60	0.81	+0.019	0.098	0.22	−0.009	60
bayesian	0.283	0.80	0.99	− 0.004	0.072	0.47	−0.028	40
futuresim	0.134	0.00	0.79	+ 0.046	0.067	− 0.14	+ 0.003	80
react	0.182	0.60	0.75	+0.019	0.066	0.27	−0.011	60

Table 2: E1 headline metrics ($k=5$ both conditions; pooled over the pair). Bold marks, per column, the value most indicative of anchoring or of its absence.

Read the futuresim row across. It moves least, never once raised both beliefs after the semifinals, carries the largest anchor gap, is the only harness with a positive lag advantage, sits at the 80th percentile of its own fresh-eyes distribution, and its update gain is *negative*. Every one of those is the signature of tracking yesterday's evidence-conditioned belief rather than today's. **bayesian** is the opposite: the best coherent tracker, and unbiased against its own fresh-eyes belief. **Every update gain is below 1**—conservatism is universal, and one harness moves the wrong way.

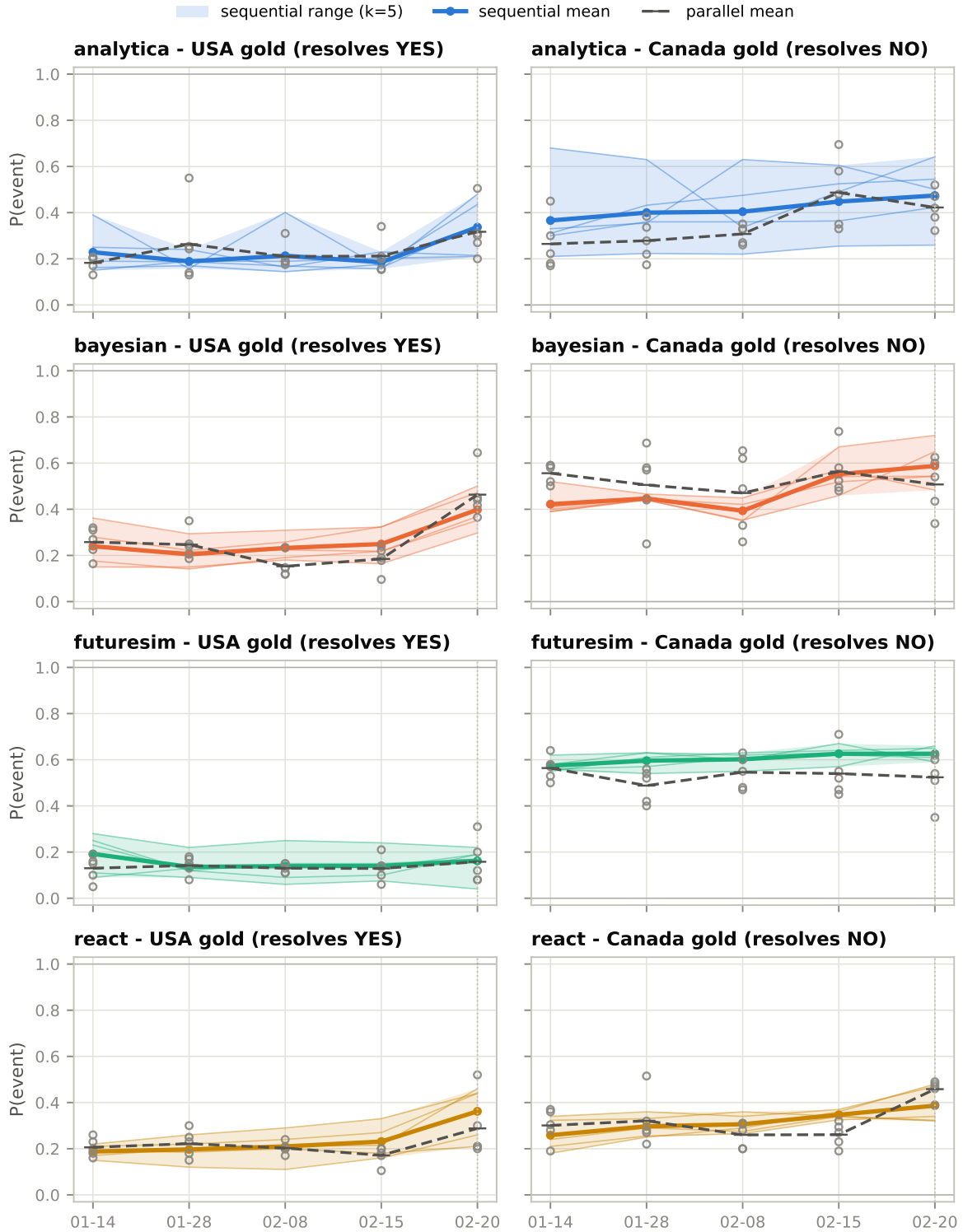


Figure 5: Belief flow, all four shapes on the paired questions. Shaded band and bold line: range and mean of the five SEQ rollouts (history carried). Open dots and dashed line: the five PAR one-shot draws per date and their mean (fresh eyes on the same evidence). Dotted vertical: semifinal date. **futuresim**-Canada's band rides above its fresh-eyes mean (anchoring) and **futuresim**-USA barely responds even in PAR (procedure-level conservatism); **bayesian** hugs its fresh-eyes mean in both panels.

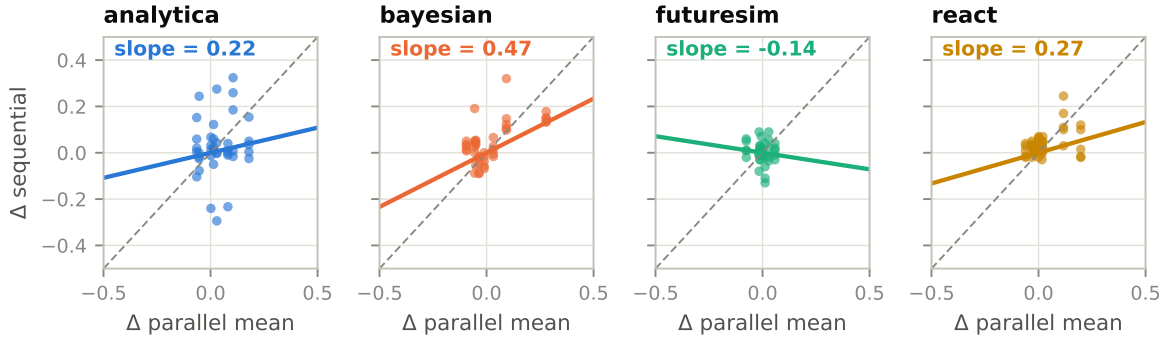


Figure 6: Update gain. Each dot is one date-step: x , movement of the fresh-eyes mean; y , movement of a sequential rollout. Dashed: $y=x$. All fitted slopes are below 1; **futuresim**'s is negative.

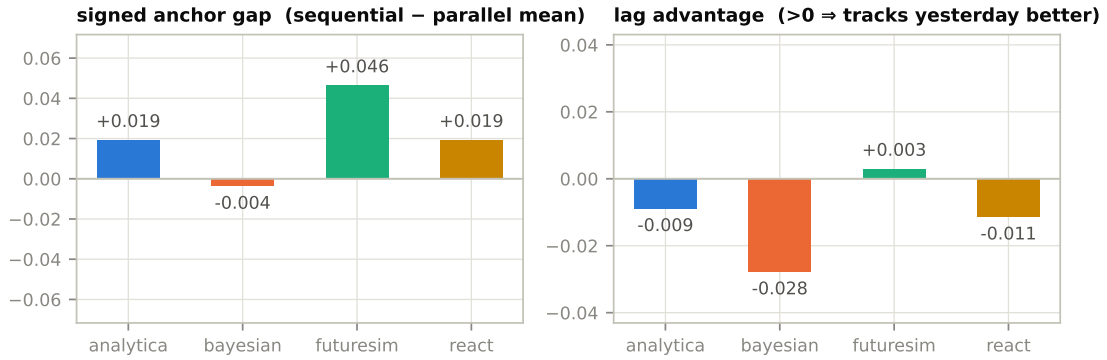


Figure 7: Anchoring summary. Left: signed anchor gap. Right: lag advantage; only **futuresim** is positive.

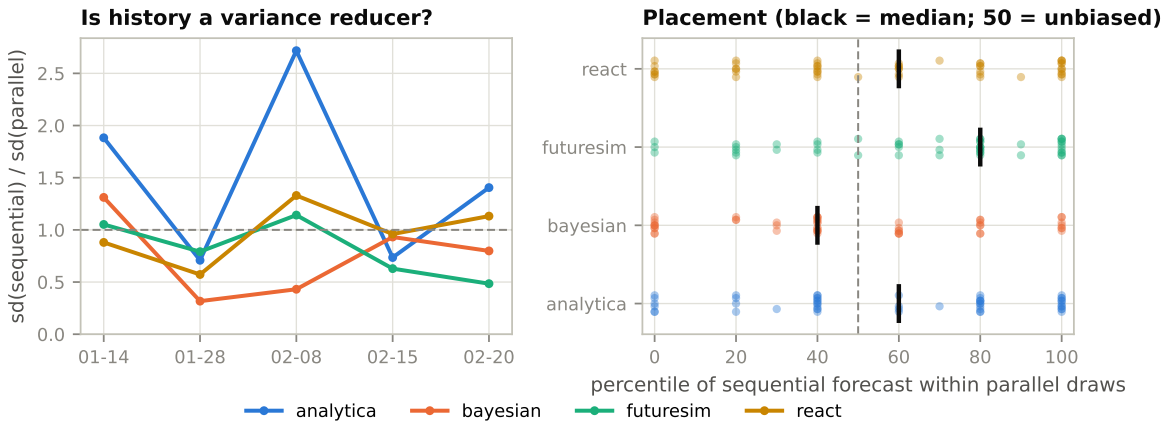


Figure 8: Left: stabilisation ratio $sd(SEQ)/sd(PAR)$. History acts as a variance reducer where the ratio sits below 1; **analytica**'s ratio above 1 on several dates is path dependence—rollouts diverge because their own early draws differed. **Right:** placement of every sequential forecast within its date's fresh-eyes distribution; **futuresim**'s median sits at the 80th percentile (anchored high), **bayesian**'s at the 40th.

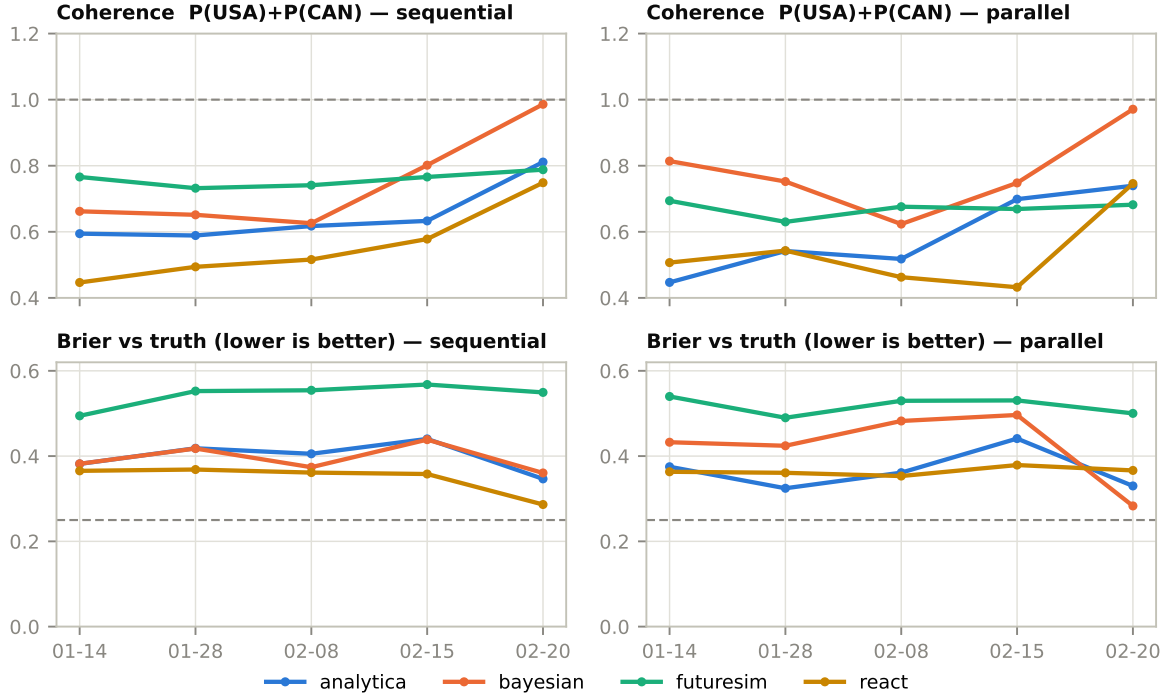


Figure 9: Coherence and accuracy by condition. Top: $P_{\text{USA}}+P_{\text{CAN}}$ should approach 1 by the semifinal (dashed). Bottom: plain Brier against the resolved outcomes, pooled over the pair; the dashed line is 0.25, the constant- $\frac{1}{2}$ forecaster.

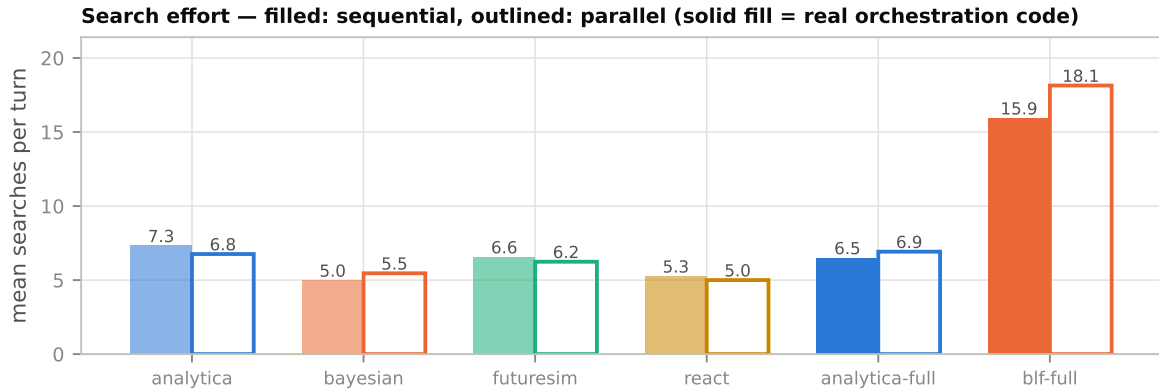


Figure 10: Search effort across both tiers. Memory does not substitute for retrieval at this scale. *analytica*'s per-sub-proposition grounding makes it the heaviest Tier-A searcher; *blf-full*'s $K=3$ trials per day make it by far the heaviest overall.

8 Experiment 1R: Replay with Real Orchestration Code

Identical model, corpus, date gate, questions and protocols. The only difference is whether the source system’s structure is *requested by a prompt* or *enforced by code*.

lineage	tier	$ \Delta _{\text{tot}}$	semi-rise	sum@last	gap	gap	gain	PIT _{med}	srch SEQ/PAR
Analytica	shape	0.331	0.60	0.81	+0.019	0.098	0.22	60	7.3/6.8
	full	0.093	0.20	1.04	+0.040	0.089	0.05	60	6.5/6.9
BLF	shape	0.283	0.80	0.99	−0.004	0.072	0.47	40	5.0/5.5
	full	0.067	0.60	0.68	+0.007	0.040	−0.17	60	15.9/18.1

Table 3: Shape vs. real orchestration, identical protocols and metrics as Table 2 (shape rows repeated from there). The comparison is confounded only by what it measures: the two tiers share model, corpus, questions and protocols, and differ in whether the source system’s structure is requested or enforced.

8.1 What the real systems change

Rigidity is architectural. The prompt-shape results *understate* the source systems’ conservatism. Real Analytica moves $3.5\times$ less than its shape ($|\Delta|_{\text{tot}}$ 0.093 vs 0.331) and its update gain collapses from 0.22 to 0.05: unchanged leaves are re-grounded only against news since they last looked, they are handed their own prior value as an anchor, and parent weights are reused unless children change—so the composed belief *cannot* respond to evidence that does not survive the edit pass. This also answers a methodological suspicion raised against the shape experiment: that a single-loop **analytica** was too easy to run to be the real thing. It was. The real pipeline behaves differently, in the direction the architecture predicts. Likewise **blf-full**: logit-space averaging over K trials with variance shrinkage is a low-pass filter applied *after* the model reasons, and its day-to-day movement (0.067) is the smallest measured in this report—while its prompt shape, freed of the aggregation arithmetic, was Tier A’s best evidence-tracker at 0.47.

Coherence can come from the wiring. **analytica-full** is the only harness in either tier whose paired beliefs sum to ≈ 1 (1.04 at the last date, 0.93–0.97 before) *without ever seeing the two questions together*: decomposition plus linear synthesis yields near-additive beliefs as a side effect of the architecture. **blf-full** sums to 0.68—the [0.05, 0.95] clamp and the shrinkage pull both members of the pair toward the interior.

Stability of the belief distribution. **analytica-full**’s stabilisation ratio sits at 0.27–0.63 through mid-window and reaches 1.04 only at the end; **blf-full**’s is 1.07–1.32—carrying the belief state does not reduce cross-rollout variance at all, consistent with the carried JSON acting as a strong prior that each rollout drags along its own path.

Accuracy caveat, stated plainly. On this pair *neither* real system beat the constant- $\frac{1}{2}$ forecaster (pooled Brier 0.31–0.44 everywhere, against 0.25). **analytica-full**’s sequential Brier worsens monotonically ($0.371 \rightarrow 0.392 \rightarrow 0.408 \rightarrow 0.415 \rightarrow 0.444$) while its parallel Brier improves ($0.375 \rightarrow 0.358$). These are $n=2$ questions and $k=5$ rollouts: **the dynamics are the result**—gains, gaps, stabilisation ratios and the shape-versus-real difference, all within-system comparisons on identical evidence. The Brier levels are reported because omitting them would be dishonest, not because they establish anything.

What is not yet covered. FutureSim’s own simulator is integrated and passes a full structural dry run, but its live run is pending (§12). **react** needs no “full” variant: ReAct is a prompt pattern over a tool loop, and §4.1’s **react** *is* its source system.

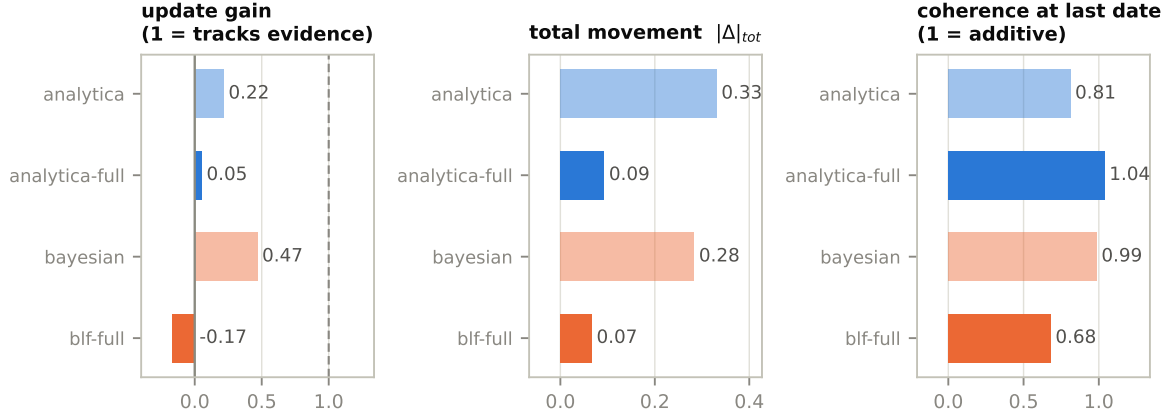


Figure 11: The headline of E1R. Same metric, both tiers, both lineages. Translucent bars: prompt shape. Solid: real orchestration code.

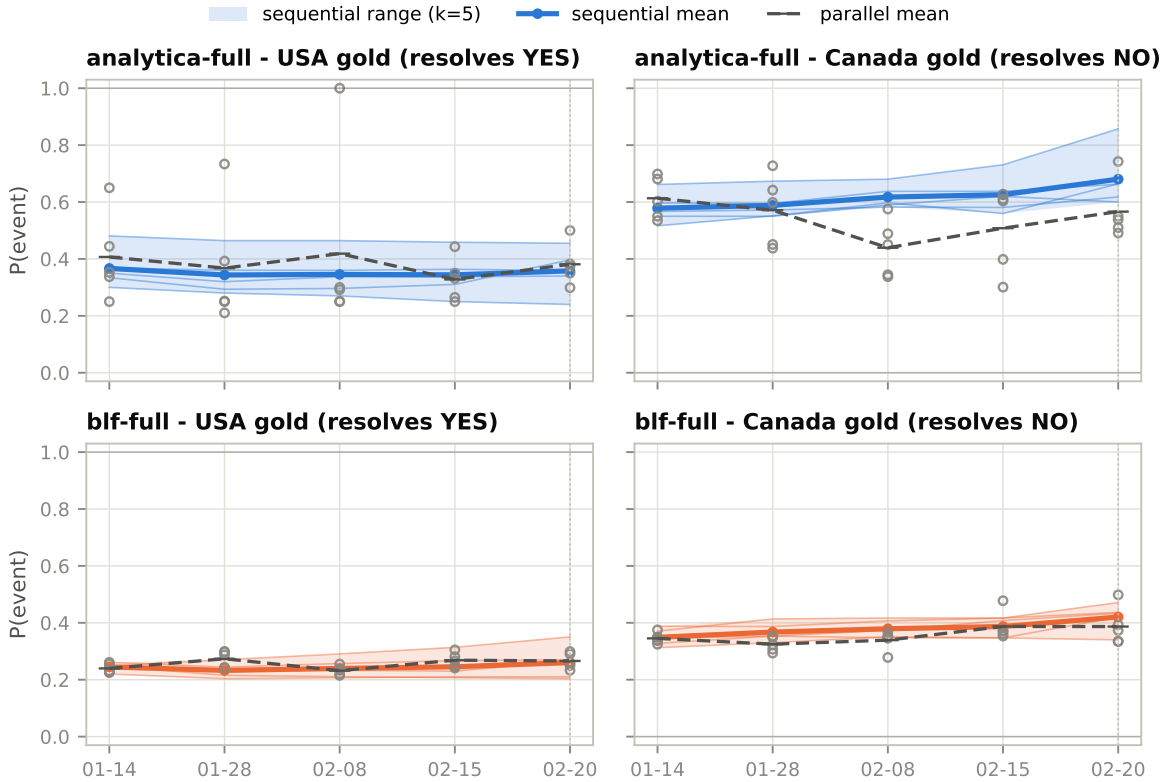


Figure 12: Belief flow of the real systems (same encoding as Fig. 5; note the full 0–1 vertical scale). **analytica-full**’s carried belief on Canada climbs steadily while its own fresh-eyes mean dips to 0.44 on 02-08 and recovers—history holding the belief above what that day’s evidence alone supported. On USA its sequential mean is nearly flat across five weeks of news. **blf-full** is tightly determined in both conditions (fresh-eyes sd ≈ 0.03) and drifts in small increments.

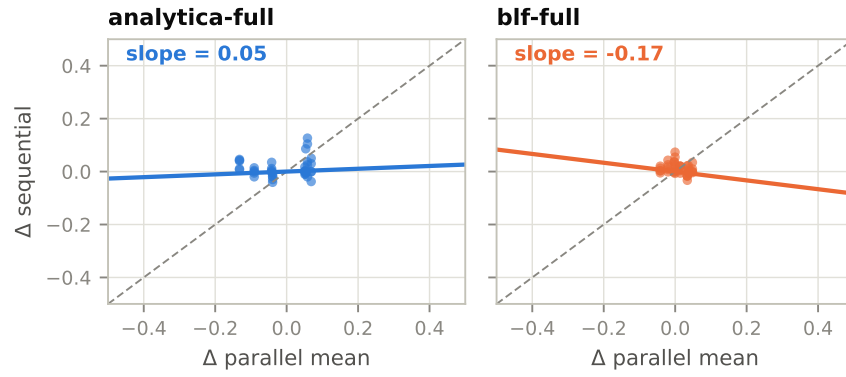


Figure 13: Update gain of the real systems. Real Analytica’s slope of 0.05 means a fresh-eyes movement of 10 points moves the carried belief by half a point. **blf-full**’s negative slope should be read with its small x -range: its fresh-eyes mean rarely moves more than ± 0.06 , so the sign is weakly identified; the robust statement is that its sequential movement is *uncoupled* from evidence movement.



Figure 14: Real-tier diagnostics. **Top left:** stabilisation ratio. *analytica-full* sits far below 1 through mid-window—carrying the tree strongly reduces cross-rollout variance—while *blf-full* sits above 1 throughout, so carrying the belief state does not reduce variance at all. **Top right:** placement of every sequential forecast within its date’s fresh-eyes distribution. **Middle:** coherence, $P_{\text{USA}} + P_{\text{CAN}}$, which should approach 1. **Bottom:** plain Brier against the resolved outcomes; the dashed line at 0.25 is the constant- $\frac{1}{2}$ forecaster. *analytica-full*’s sequential Brier worsens monotonically while its parallel Brier improves—on identical evidence, carrying its own tree cost it accuracy that fresh eyes recovered.

9 Experiment 2: Program Recovery

9.1 Pipeline

For every forecast turn: **step 1**, the extractor reads that day’s searches and reasoning and emits `name = value` declarations for every quantity the reasoning used—never the output, in any parameterisation (raw, odds, percent, complement all dropped within ± 0.005 ; yesterday’s forecast exempted as the legitimate anchor input); names must describe world quantities, never values. **Step 2** reconciles the per-day tables into one canonical schema (missing instance keys retried once, then excluded loudly—never zero-filled; pre-reconciliation tables preserved as the audit trail). **Step 3** synthesizes *one* program per harness–question against all (≤ 10) instances, repaired against validator violations and worst residuals for at most four attempts; then re-synthesizes *blind* on (a) sample 0 only and (b) the first three dates only, evaluated untouched on the held-out complement.

group	in-sample	sample-holdout	temporal (early→late)	reading
bayesian/USA	0.027	0.016	0.0005 → 0.003	fixed log-odds formula
bayesian/CAN	—	0.063	0.0003 → 0.002	same; full-fit failed to execute
futuresim/CAN	0.018	0.014	0.015 → 0.021	stable; either half recovers the same program
futuresim/USA	0.045	0.135	0.051 → 0.032	stable-ish
react/CAN	0.070	0.073	0.044 → 0.046	weak method
react/USA	none	0.178	0.252 → 0.487	incompressible
analytica/USA	0.108	0.274	0.073 → 0.285	structure changes daily
analytica/CAN	0.301	0.396	0.520 → 0.335	

Table 4: Program-recovery MAE. Holdout columns are blind (targets hidden from the synthesizer); bold marks MAE at or below the 0.02 tolerance. No group passes the strict gate *on the full in-sample fit* ($\text{MAE} \leq 0.02 \wedge \geq 80\%$ of instances within tolerance $\wedge 0$ failures), so in-sample fits are reported as descriptive—but two blind holdouts individually meet every threshold (**futuresim/CAN** sample-holdout: 0.014 at 80% in-tolerance, 0 failures; **bayesian** temporal: 0.002–0.003 at 100%), and the blind columns are the honest evidence throughout. Scale-up (four further questions, 16 groups, hardened rules): 10 clean, 5 marked recovery-uninformative (near-constant targets make both copy-detection and recovery ill-posed), 1 no-program.

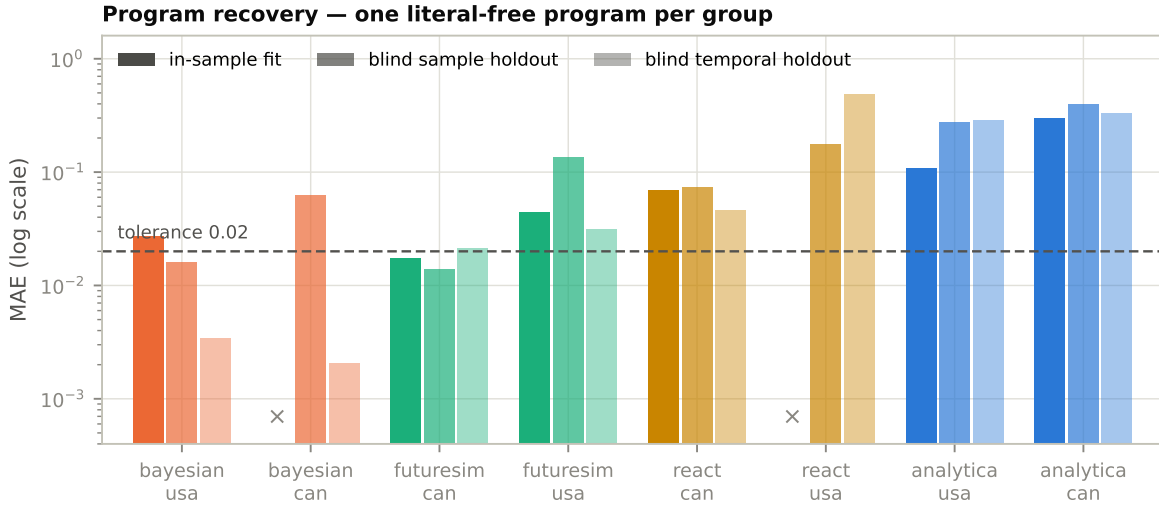


Figure 15: Recovery quality per group (log MAE; × marks “no valid program”). **bayesian**’s blind temporal holdouts reach MAE 0.002–0.003: a genuinely fixed update formula. **analytica** fits poorly under every regime—its structure changes day to day. **react**–USA admits no program at all.

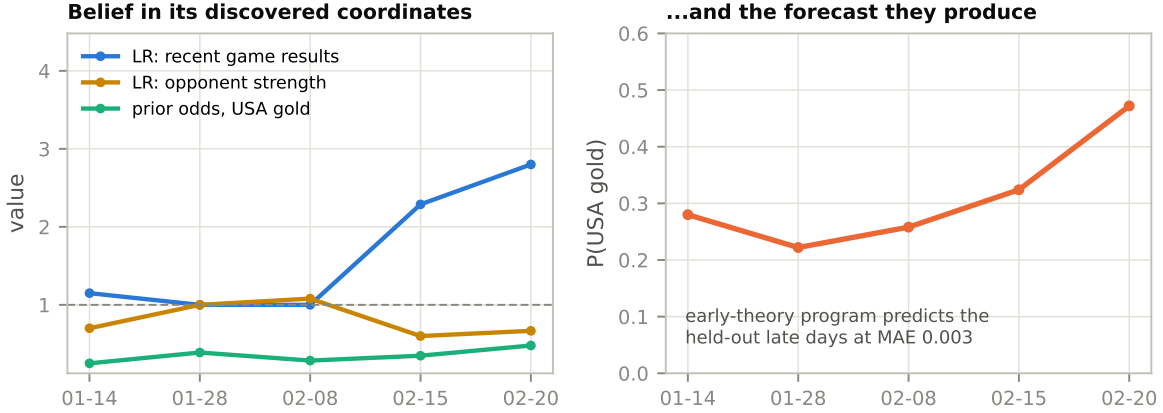


Figure 16: The belief in its discovered coordinates (*bayesian*/USA, rollout s0). Left: three recovered variables across the window—the recent-results likelihood ratio climbs $1.1 \rightarrow 2.9$ as the USA advances while the prior odds ratchet behind it. Right: the forecast those coordinates produce. The recovered programs are log-odds additive updating with a shrink-to-prior term (full-fit version in Appendix A); re-synthesized blind on the first three dates alone, the temporal variant—same family, different expression—predicts the last two days at the MAE shown.

9.2 Residual drift and theory change

Fitting one program across all dates makes *failure* informative: *analytica*/CAN’s mean |residual| grows by +0.34 from first to last date (USA: +0.05)—its method decays—while *futuresim*’s is flat or improving (CAN −0.007, USA −0.07). Function-space distance between the full-fit and the sample-0-only programs (Monte-Carlo over the observed binding box) says the same thing more sharply: *futuresim*/CAN **0.000** (identical theory from either half of the data), *bayesian*/USA 0.044, *react*/CAN 0.174, *analytica*/CAN 0.391.

10 The Rigidity Probe

Each probed harness was continued from its real end-of-run state to a probe day (02-21) with search disabled (stated as an outage) and exactly one injected, role-based news item—“Canada’s starting goaltender ruled out of the final” / the USA mirror / an irrelevant placebo. Its own recovered program supplies the referee twice over: a *single-coordinate* implied change (perturb only the targeted variable), and a *multi-coordinate* rule-implied forecast (re-extract the probe day’s full bindings from the harness’s own reasoning and execute the program). Placebo movements were ≤ 0.02 ; **all sixteen directional probes (4 groups $\times$ 2 rollouts $\times$ 2 directions) moved the correct direction.**

harness	method stable?	tracks evidence?	honours own rule?	diagnosis
<i>futuresim</i>	yes	poorly in replay	yes (gap ≤ 0.04)	rigidity lives in <i>acquisition</i> , not revision
<i>bayesian</i>	yes	good	asymmetric	compliant on good news; over-drops on bad news vs own rule (≈ -0.135)
<i>react</i>	no	yes	n/a — no channel	updates outside recoverable structure
<i>analytica</i>	no	good	no verdict possible	rewrites its method daily

Table 5: Per-harness verdicts, combining §7–§10. The one-line finding: structural stability and evidence-tracking anticorrelate, and the probe *localises* each harness’s rigidity or plasticity.

Two rows deserve reading twice. ***futuresim* is not a rule-breaker**—handed the evidence directly it is more faithful to its own method than any other harness, and its failure in replay is that it does not go

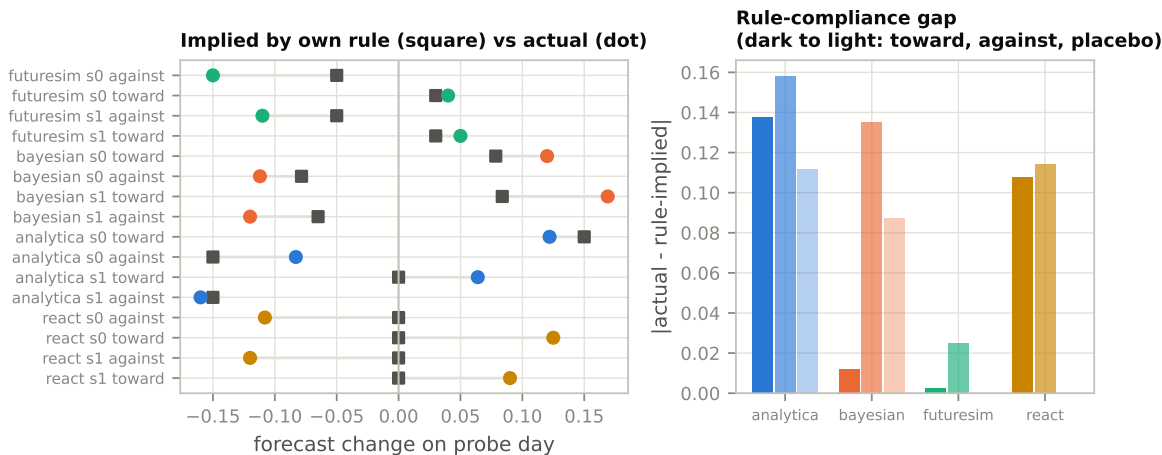


Figure 17: Left: single-coordinate implied change (square) vs actual change (dot) for every directional probe. **Right:** the multi-coordinate rule-compliance gap $|p_{\text{actual}} - p_{\text{rule}}|$ by condition. **futuresim**’s gaps are ≤ 0.04 : handed the evidence, it computes with its recovered structure almost exactly. **react**’s program implies zero response in every cell while the live harness moved ± 0.09 – 0.13 : its updates use machinery its recoverable structure does not contain.

and find the evidence. That is a fixable engineering problem and a completely different diagnosis from “the model is stubborn”. And **react**’s program implies zero response in every cell while the live harness moved ± 0.09 – 0.13 : its updates are real and correctly directed, but they travel through channels no program in our space contains.

11 Adversarial Review and Standing Audit

Mid-experiment, an independent reviewer was pointed at the entire codebase with instructions to break it. It returned 2 fatal, 7 serious, and 6 moderate findings; every fixable one was patched and re-verified before results were trusted.

The fatals. (F1) the planned “separation” metric would have punished correct forecasting given the actual event path (§3.3); (F2) the no-literal rule was semantically bypassable—constant-construction, boolean arithmetic, x/x , dead-code padding, and if-chain lookup tables that the repair loop was effectively coaching the synthesizer to write—closed by the rules of §5 plus the blind holdouts.

Serious findings worth naming. A thread-unsafe SQLite connection corrupted 32% of an early run (that run is quarantined and unused). A forecast parser could fabricate certainty from “85%” via a bare-number fallback (fixed; the strict parser reproduces all collected forecasts exactly, and the 240 already collected were retro-verified to re-parse identically). Schema reconciliation silently zero-filled missing keys, which would have invented belief values that were never stated (replaced by loud exclusion plus a preserved audit trail).

The standing audit. `scripts/audit_exp1.py` re-verifies on demand: 11/11 exploits rejected; all collected forecasts parse under the strict parser; 1,432 searches replayed with zero gate breaches; every stored program re-executes to its reported MAE; one systematic answer-copy in 320 variable tables, referenced by no program. One refinement cuts the other way and is recorded for honesty: the first copy-scan *over-flagged*, treating single-instance coincidences as contamination and missing the legitimate prev-day-forecast exemption. The criterion was tightened to “systematic across $\geq 80\%$ of instances”, and groups with near-constant targets are now marked *recovery-uninformative* rather than contaminated—a weaker claim, and the correct one.

12 Limitations

1. One question pair carries all directional claims; the scale-up set is all-YES by construction and supports only recovery statistics.
2. $k=5$ per cell bounds distributional claims; PIT placement has 5-draw granularity.
3. Draws are ordinary decoding variation (current reasoning endpoints reject temperature control), so sampling spread cannot be widened or narrowed deliberately.
4. Stated beliefs only: all measurements are consistency properties of verbalized probabilities under intervention, not claims about internal representations.
5. Retrieval is BM25, weaker than FutureSim’s hybrid index—held constant across harnesses, but absolute accuracy is a lower bound.
6. The extractor knows the outcomes; the audit bounds but does not eliminate this channel.
7. Program recovery inherits the expressiveness of the program space; “incompressible” means no program *in this space* within four repair attempts.
8. Experiments 1, 2 and the probe use prompt-level shapes on a shared loop (§4.1), not the source systems. §8 closes this for the replay experiment on two of the three systems—and confirms the concern was material: the real architectures are substantially more rigid than their shapes. **It remains open for futuresim, which is the worst place for it to remain open**, because futuresim carries more of this report’s distinctive claims than any other harness: the only negative Tier-A update gain, the only positive lag advantage, the 80th-percentile placement, the semi-rise of zero, the cleanest program recovery, and the rule-compliance result behind “rigidity lives in acquisition, not revision”. Every one of those is a statement about a prompt shape. The gap to the real thing is not small: their **basicAgent** manages memory through four CRUD tools rather than a 200-word text block, queries the question table by writing Python, submits a distribution over outcomes it generates itself, and—most consequentially for us—*chooses when to update* via **next_day**, whereas our shape is forced to emit a forecast on every date. FutureSim’s own anchoring finding is partly about agents declining to update at all, a behaviour our shape cannot exhibit by construction. The direction of the correction is therefore unpredictable, unlike the Analytica and BLF cases where the architecture could only add rigidity. The integration needed to close this gap is built and passes a structural dry run (§4.2); only the paid run is outstanding. E2 and the probe likewise remain prompt-tier throughout.
9. **blf-full**’s negative update gain is weakly identified (its fresh-eyes mean moves little; Fig. 13), and $K=3$ trials rather than the paper’s $K=5$, with calibration omitted for lack of in-window training data, bound fidelity to the BLF paper.

13 Conclusion and Future Work

Treating a forecasting agent’s reasoning as a recoverable, literal-free program turns belief dynamics from a metaphor into a measurement: beliefs become trajectories in discovered coordinates, updates become testable against the agent’s own rule, and rigidity decomposes into acquisition, procedure, history and revision components that this report measures separately. Running the real orchestration code of two source systems then adds an axis the prompt tier cannot see—prompts understate architectural rigidity, and the shape-versus-real gap is itself the measurement.

The single most valuable experiment left undone is program recovery on the *real* Analytica traces, because it is a known-rule validation: **analytica-full**’s composition rule is literally $\beta_0 + \sum \beta_i p_i$, computed in code and logged in every trace. If the recovery pipeline is sound it should find exactly that; if it finds something else, the pipeline is wrong and we would know. No other experiment in this report has ground truth to check itself against. After that: FutureSim’s simulator live on the same sandbox; paired NO-questions and longer date grids; per-search-call budgets; and program recovery on the parallel condition—does removing history delete the anchor variables from the schema entirely?

References

- [1] *Analytica: Soft Propositional Reasoning for Robust and Scalable LLM-Driven Analysis*. arXiv:2604.23072, 2026.
- [2] *FutureSim: Replaying World Events to Evaluate Adaptive Agents*. arXiv:2605.15188, 2026.
- [3] *Agent-BRACE: Decoupling Beliefs from Actions in Long-Horizon Tasks via Verbalized State Uncertainty*. arXiv:2605.11436, 2026.
- [4] *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR, 2023.
- [5] *Agentic Forecasting using Sequential Bayesian Updating of Linguistic Beliefs*. arXiv:2604.18576, 2026.
- [6] *Verification of forecasts expressed in terms of probability*. Monthly Weather Review, 1950.

A A Recovered Program

The `bayesian/USA` program (full-fit). The blind variants recover the same log-odds-additive family, though not this exact expression. Note the absence of literals; every quantity is a step-1 variable or a named constant. The schema contained an odds-form copy of the answer (`posterior_odds_usa_gold`); the program does not reference it.

```
def forecast():
    p0 = prior_probability_usa_gold
    o0 = prior_odds_usa_gold
    ln_prior_odds = log(o0)
    ln_lr_total = log(lr_roster_strength_positive)
    ln_lr_total += log(lr_opponent_canada_strength)
    ln_lr_total += log(lr_injury_and_replacements)
    ln_lr_total += log(lr_recent_game_results)
    ln_lr_total += log(lr_nhl_participation_logistics)
    ln_lr_total += log(lr_venue_or_media_sentiment)
    ln_post_odds = ln_prior_odds + ln_lr_total
    p_bayes = logistic(ln_post_odds)
    logit_p0 = log(p0) - log(ONE - p0)
    gap = ln_post_odds - logit_p0
    tight = ONE / (ONE + abs(gap))
    w = (HALF + tight) * HALF
    return p0 + w * (p_bayes - p0)
```

B Harness Prompts, Verbatim

B.1 Tier A: the shared preamble

You are forecasting a real future event.

The current simulation date is {date}. You know nothing about events after this date. Use `search_news` to gather evidence; it can only return articles published on or before {date}.

Write your reasoning out explicitly as you go -- state the quantities you are weighing and the numbers you assign them. Someone will later have to reconstruct exactly how you arrived at your number.

End your final message with a line of exactly this form:

FORECAST: <probability between 0 and 1>

B.2 Tier A: the four shape suffixes

[react]

Work in a simple loop: think about what you need, search for it, read what comes back, think again. Stop when further searching would not change your answer.

[futuresim]

FORECASTING GUIDELINES (follow them in order):

1. Restate the resolution criterion precisely. What exactly must happen?
2. Set a base rate from reference-class outcomes before looking at news.
3. Search for evidence. Run several distinct, targeted queries, not one broad one. Look for disconfirming evidence specifically.
4. Weigh evidence against the base rate. State how much each item moves you and why.
5. Check yourself for overconfidence and for anchoring on your previous forecast. A forecast that never moves is as suspect as one that thrashes.
6. Give a calibrated probability.

MEMORY. Your memory from previous days is below. Before your final answer you MUST output an updated memory block, in this exact form:

```
<memory>
BASE RATE: ...
KEY EVIDENCE: ...
WHAT WOULD CHANGE MY MIND: ...
LAST FORECAST AND WHY: ...
</memory>
```

Keep it under 200 words. It is all you will carry to the next day.

[analytica]

Work in three explicit stages, labelled.

STAGE 1 - ANALYSE. Break the question into 3 to 5 sub-propositions that are independent of each other and each individually checkable. Number them P1, P2, ... Do not search yet.

STAGE 2 - GROUND. For each sub-proposition in turn, search for evidence and assign it a probability. Write P1 = <p> with a one-line justification.

STAGE 3 - SYNTHESISE. Assign each sub-proposition a weight showing how much it contributes to the overall question, state the combination you are performing with its numbers written out, and compute the result. Then give FORECAST.

[bayesian]

Maintain an explicit belief state and update it in the open.

Your belief state from previous days is below. Each day, in this order:

PRIOR: state your current probability and the one-sentence summary of the evidence it rests on.

EVIDENCE: search, then state what is new since your last update.

LIKELIHOOD: for each new item, state how much more likely it is if the event happens than if it does not -- an explicit ratio, e.g. "about 3x more likely under YES".

POSTERIOR: apply the update to your prior odds and state the result.

Then output an updated belief state in this exact form:

```

<belief>
P: <probability>
EVIDENCE SUMMARY: ...
STRONGEST COUNTER-EVIDENCE: ...
</belief>

```

Then give FORECAST.

B.3 Tier B: the real-harness agent prompts

```

[analytica-full :: ANALYZER, day 1]
You are the Analyzer in the Analytica forecasting system.
You expand a proposition tree. Each proposition must be atomic, checkable, and
child propositions of one parent should be as independent of each other as
possible. Leaves should be simple enough that a search-equipped analyst can
score them from news evidence.
Respond with JSON only:
{"add": [{"parent": "<existing node id>", "statement": "<new proposition>"}, ...],
 "done": true/false}
Add 2-4 children per expansion; set done=true when the leaves are atomic enough
or the leaf budget is reached.

[analytica-full :: ANALYZER, edit pass on later dates]
You are the Analyzer in the Analytica forecasting system, revisiting an existing
proposition tree on a new day. New information may have made some propositions
obsolete or new drivers relevant. Most days need no change.
Respond with JSON only:
{"add": [...], "remove": ["<leaf node id>", ...], "done": true}
Only add or remove if genuinely warranted; otherwise return empty lists.

[analytica-full :: GROUNDER, query phase]
You are a Grounder agent in the Analytica forecasting system. Your job: assess
ONE atomic proposition against news evidence and output a soft truth value.
Today is {date}. You may request searches of a dated news archive (articles up
to today only). First reply with up to {max_q} search queries as JSON:
{"queries": ["...", "..."]}
After receiving results you will be asked to conclude.

[analytica-full :: GROUNDER, delta note -- appended on every date after the first]
You previously assessed this proposition on {prev_date} as p_true={prev_p} with
report: {prev_report}
Search results below cover ONLY the period since then. If nothing material
changed, keep your value close and say so.

[analytica-full :: GROUNDER, conclude phase]
You are a Grounder agent in the Analytica forecasting system. You have already
gathered the evidence shown below for ONE atomic proposition. Today is {date};
the evidence contains only articles up to today.
Assess the proposition's soft truth value from the evidence (and general
pre-{date} knowledge). Do NOT request searches. Output JSON only:
{"p_true": <0..1>, "report": "<3-6 sentence evidence summary with dates>"}

[analytica-full :: SYNTHESIZER]
You are the Synthesizer in the Analytica forecasting system. Given a parent
proposition and its scored children, output linear-combination weights expressing
how the children's truth values compose into the parent's:

$$p_{\text{parent}} = b_0 + \sum_j \beta_j p_{\text{child}_j}$$

Respond with JSON only:

```

{"b0": <float>, "betas": {"<child_id>": <float>, ...}, "report": "..."}
 Prefer b0 >= 0, betas >= 0, and b0 + sum(betas) <= 1 so the result is a probability.

[blf-full :: the single agent prompt, used at every step of every trial]
 You are a forecasting agent. Question: {question}
 Today is {date}; you know nothing after this date. Resolution: {resolution}.

At every step output JSON only, with BOTH an action and your updated belief:
 {"action": {"type": "search", "query": "..."} OR {"type": "submit", "p": <0..1>},
 "belief": {"p": <0..1>, "confidence": "low|medium|high",
 "evidence_for": [...], "evidence_against": [...],
 "open_questions": [...]}}

Search returns dated news articles (up to today only). Submit when further search would not change your belief. You have at most {tmax} steps.

C Per-date Series

Table 6 gives every per-date series behind the figures of §7 and §8, exactly as stored in each run directory’s seqpar_metrics.json.

harness	series	01-14	01-28	02-08	02-15	02-20
analytica	stabilisation	1.88	0.71	2.72	0.74	1.41
	coherence (SEQ)	0.594	0.589	0.617	0.633	0.811
	Brier (SEQ/PAR)	.382/.375	.418/.325	.405/.361	.440/.441	.347/.330
bayesian	stabilisation	1.31	0.32	0.43	0.93	0.80
	coherence (SEQ)	0.662	0.651	0.626	0.802	0.986
	Brier (SEQ/PAR)	.382/.432	.418/.424	.374/.482	.439/.496	.360/.283
futuresim	stabilisation	1.05	0.79	1.14	0.63	0.48
	coherence (SEQ)	0.766	0.732	0.741	0.766	0.788
	Brier (SEQ/PAR)	.494/.540	.552/.490	.554/.530	.568/.531	.549/.500
react	stabilisation	0.88	0.57	1.33	0.96	1.13
	coherence (SEQ)	0.447	0.494	0.516	0.578	0.749
	Brier (SEQ/PAR)	.365/.363	.368/.361	.361/.353	.358/.379	.286/.366
analytica-full	stabilisation	0.54	0.37	0.27	0.63	1.04
	coherence (SEQ)	0.95	0.93	0.96	0.97	1.04
	Brier (SEQ/PAR)	.371/.375	.392/.388	.408/.312	.415/.366	.444/.358
blf-full	stabilisation	1.27	1.07	1.32	1.07	1.14
	coherence (SEQ)	0.59	0.60	0.62	0.63	0.68
	Brier (SEQ/PAR)	.346/.349	.363/.317	.362/.353	.361/.344	.364/.346

Table 6: Per-date series, both tiers. Stabilisation is $\text{sd}(\text{SEQ})/\text{sd}(\text{PAR})$; coherence is $P_{\text{USA}} + P_{\text{CAN}}$; Brier is the plain quadratic score pooled over the pair (0.25 is the constant- $\frac{1}{2}$ forecaster).

D Cost Ledger and Reproduction

run	model calls	input tokens	output tokens	wall clock
E1R sequential + parallel (Tier B)	3,229	6,388,301	1,460,431	41 min

Table 7: Tier-B cost, against a hard 4,000-call cap enforced in the model wrapper (BudgetExceeded). Harness model gpt-5-mini; 4 worker threads; zero turn errors across 200 forecasts. Usage is written to results/exp1_real/usage.json by the runner.

Running `python bootstrap.py` in a fresh clone of <https://github.com/DDI-droid/belieflens> downloads the corpus (~412MB), builds the index, and runs the verification battery. Set `OPENAI_API_KEY` in `.env` (gitignored; see `.env.example`) or the environment; the harnesses need `gpt-5-mini` and the extractor `gpt-5.2`.

`scripts/` contains every stage: `run_exp1.py` (SEQ/PAR/E2), `run_real_e1.py` (§8 on the real harnesses in `belieflens/real_harnesses.py`), `audit_exp1.py`, `beliefdyn.py`, `run_probe.py`, `probe_rule_check.py`, `make_charts.py`. `futuresim_integration/` documents the pinned FutureSim-simulator integration and reproduces it from a clean clone. `build_report.ps1` regenerates every figure, compiles this document, and renders page images. All collected data, configs and logs ship in `results/`.